Name: Kedar Somnath Dhotre
PRN: 23510090

Experiment no 8

Title: Comparative Study of OpenMP and MPI for Parallel Applications

1. **Aim:**

To implement selected computationally intensive applications using both OpenMP and MPI, and experimentally compare their performance, scalability, memory model, communication overhead, and suitability for different HPC workloads.

You will:

- Implement the same computational problem using OpenMP and MPI.
- Apply shared-memory and distributed-memory parallel programming models.
- Compare execution time, speedup, efficiency and scalability.
- Analyse the effect of increasing threads/processes.
- Identify the advantages and limitations of OpenMP and MPI for different applications.
- Relate the experimental observations to real-world HPC and AI workloads.

2. **Why?**

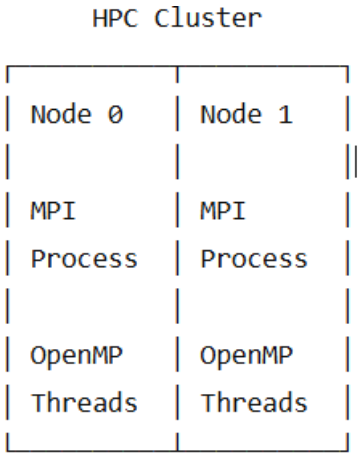
OpenMP and MPI are two fundamental programming models used in HPC, but they solve parallel programming problems differently. The same problem can often be solved using both approaches, but their performance can be different. Therefore, an HPC programmer should not only know how to parallelize a program, but also understand: Which parallel programming model is appropriate for a given problem and architecture?

3. **HPC in Practice**

Modern HPC applications frequently use combinations of programming models.

For example, a cluster node may contain multiple CPU cores.

A distributed application can use:



This is known as hybrid MPI + OpenMP programming.

Applications such as:

- Weather and climate simulation
- Computational fluid dynamics
- Molecular dynamics
- Image processing
- Deep learning
- Scientific simulations

can use these programming models.

4. Learning Focus

Concept	OpenMP	MPI
Memory model	Shared memory	Distributed memory
Parallel unit	Thread	Process
Communication	Shared variables	Message passing
Data sharing	Implicit/shared	Explicit
Scalability	Within shared-memory system	Across multiple nodes
Synchronization	Critical, barrier, atomic, reduction	Send/Receive, collective communication

Programming complexity	Generally lower	Generally higher
Multi-node execution	Limited	Well suited

5. Pre-Lab Activity

Before starting the implementation, answer:

1. What is the difference between a thread and an MPI process?
   - Thread: A lightweight execution unit inside a process. Threads share the same memory.
   - MPI Process: An independent process with its own separate memory.
   - Threads communicate through shared variables, while MPI processes communicate using MPI messages.
   - Example: OpenMP → multiple threads in one program. MPI → multiple processes running the program

2. What is shared-memory parallelism?
   Shared-memory parallelism is a method where multiple threads/processors use the same memory space.
   - Data can be accessed directly by all threads.
   - Communication is relatively fast.
   - OpenMP is commonly used for shared-memory parallelism.

   Example: Multiple threads working on different parts of the same array.

3. What is distributed-memory parallelism?

   Distributed-memory parallelism is a method where each process has its **own private memory**.
   - Processes cannot directly access each other's memory.
   - They communicate by sending and receiving messages.
   - **MPI** is commonly used for distributed-memory parallelism.

   Example: A matrix is divided among processes running on different computers.

4. How does OpenMP distribute loop iterations?
   - OpenMP divides the iterations of a loop among multiple **threads**.

     For example:

```
#pragma omp parallel for
for(i = 0; i < 100; i++)
```

- If there are 4 threads, OpenMP can divide the 100 iterations among the 4 threads.
- So instead of one thread doing all 100 iterations, each thread handles a portion.

5. How does MPI distribute data among processes?

- MPI distributes data by explicitly sending portions of data to different processes.
- For example, with:

  MPI_Scatter()

- the **root process** divides the data into portions and sends one portion to each process.

  Example:

  Data: 1 2 3 4 5 6 7 8
  Process 0: 1 2
  Process 1: 3 4
  Process 2: 5 6
  Process 3: 7 8

6. What is communication overhead?

Communication overhead is the **extra time required to exchange data or coordinate between parallel workers**.

In MPI, this mainly includes:

- Sending data
- Receiving data
- Synchronization
- Network communication

If communication takes too much time, the benefit of parallelism can decrease.

7. Why can an MPI program be slower than OpenMP for some problems?

- MPI can be slower because MPI processes need to communicate by sending messages.
- OpenMP threads share memory, so they can access common data directly.
- Therefore: Small problem + lots of communication → MPI may be slower.
- For a single computer with shared memory, OpenMP can sometimes have lower communication overhead.

8. Why is MPI more suitable for multi-node systems?

MPI is designed for **distributed-memory systems**, where different processes can run on different computers/nodes.

Each node has its own memory, and MPI allows the nodes to communicate through a network.

Node 1 Node 2 Node 3

Process 0 Process 1 Process 2

Memory Memory Memory

\ | /

MPI Communication

Therefore, MPI can scale across **multiple computers**, unlike traditional OpenMP, which is primarily designed for shared memory within a system.

9. What is strong scaling?

Strong scaling measures how the execution time changes when we increase the number of processors while keeping the **problem size fixed**.

Example:

Problem size = 1,000,000 elements

1 process → 10 seconds

2 processes → 5 seconds

4 processes → 2.5 seconds

The problem remains the same; only the number of processors increases.

Goal: Finish the same problem faster.

10. What is weak scaling?

Weak scaling measures how the execution time changes when we increase both the number of processors and the problem size proportionally.

Example:

1 process → 1 million elements

2 processes → 2 million elements

4 processes → 4 million elements

Each processor gets approximately the same amount of work.

Goal: Handle a larger problem without significantly increasing execution time.

Section A – Task 1: Matrix–Matrix Multiplication

Problem Statement

Matrix multiplication is selected because it is computationally intensive and provides sufficient workload for meaningful performance comparison.

Task 1A – Sequential Implementation

Suggested sizes:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

Task 1B – OpenMP Implementation

Parallelize the matrix multiplication using OpenMP.

Use:

#pragma omp parallel for

Investigate different numbers of threads:

- 2
- 4
- 8
- Maximum available threads

Measure execution time.

Task 1C – MPI Implementation

Implement the same matrix multiplication using MPI.

Use **row-wise decomposition**.

Suggested strategy:

1. Process 0 initializes matrices (A) and (B).
2. Distribute rows of (A) among MPI processes.
3. Broadcast matrix (B).
4. Each process computes its assigned rows.
5. Gather the resulting rows of (C).

Use appropriate MPI collective operations.

Task 1D – Compare OpenMP and MPI

For the same matrix sizes, compare:

- Sequential
- OpenMP
- MPI

Prepare:

Matrix Size	Parallel Model	Threads/Processes	Execution Time (s)	Speedup	Efficiency
500	Sequential	1	**0.052589**	**1.000**	**100.00%**
500	OpenMP	2	**0.032918**	**1.598**	**79.88%**
500	OpenMP	4	**0.036441**	**1.443**	**36.08%**

500	OpenMP	8	**0.035116**	**1.498**	**18.72%**
500	MPI	2	**0.029790**	**1.765**	**88.27%**
500	MPI	4	**0.029854**	**1.762**	**44.04%**
500	MPI	8	**0.023476**	**2.240**	**28.00%**

Matrix Size	Parallel Model	Threads/Processes	Execution Time (s)	Speedup	Efficiency
1000	Sequential	1	0.462690	1.000	100.00%
1000	OpenMP	2	0.346864	1.334	66.70%
1000	OpenMP	4	0.288236	1.605	40.13%
1000	OpenMP	8	0.251331	1.841	23.01%
1000	MPI	2	0.453253	1.021	51.04%
1000	MPI	4	0.314938	1.469	36.73%
1000	MPI	8	0.264096	1.752	21.90%

Matrix Size	Parallel Model	Threads/Processes	Execution Time (s)	Speedup	Efficiency
2000	Sequential	1	18.496486	1.000	100.00%
2000	OpenMP	2	11.929673	1.550	77.52%
2000	OpenMP	4	7.706135	2.400	60.01%
2000	OpenMP	8	4.172612	4.433	55.41%
2000	MPI	2	12.093108	1.530	76.48%
2000	MPI	4	6.738585	2.745	68.62%
2000	MPI	8	3.950362	4.682	58.53%

Analyze

1. Which implementation has lower execution time?

   For the 500 × 500 matrix in this experiment, the lowest measured execution time was obtained by MPI with 8 processes, at:

   $$0.023476 \text{ seconds}$$

   The measured times were:

   Sequential 0.052589 s

   OpenMP 2 0.032918 s

   OpenMP 4 0.036441 s

   OpenMP 8 0.035116 s

   MPI 2 0.029790 s

   MPI 4 0.029854 s

   MPI 8 0.023476 s

   So for this particular experiment, MPI with 8 processes produced the lowest measured time.

2. How does performance change as threads/processes increase?

   Your results show that performance does **not always improve proportionally** as the number of workers increases.

OpenMP

2 threads → 0.032918 s

4 threads → 0.036441 s

8 threads → 0.035116 s

12 threads → 0.036319 s

The 2-thread execution was actually faster than the 4-, 8- and 12-thread executions.

This can happen because the matrix is relatively small and the overhead of creating, scheduling and coordinating more threads becomes significant.

MPI

2 processes → 0.029790 s

4 processes → 0.029854 s

8 processes → 0.023476 s

The 8-process MPI run was fastest among the MPI configurations tested.

Conclusion: Increasing the number of threads/processes does not guarantee proportional speedup.

3. What is the communication overhead in MPI?

   MPI communication overhead is the time required for processes to exchange data and synchronize.

   In our MPI matrix multiplication program, communication occurs mainly through:

   MPI_Scatterv()

   which distributes rows of matrix A,

   MPI_Bcast()

   which sends matrix B to all processes,

   and

   MPI_Gatherv()

   which collects the calculated rows of matrix C.

   Therefore, MPI execution involves both: computation,communication and syncronization

   For a small matrix, this communication can represent a relatively larger portion of the total execution time.

4. Why does OpenMP have lower communication overhead on a shared-memory system?

   OpenMP uses threads within the same shared-memory system.

All threads can directly access the same: A B C

matrices.

They do not need to explicitly send the matrices between separate processes using MPI communication operations.

MPI processes, on the other hand, have separate memory spaces and need operations such as:

MPI_Scatterv

MPI_Bcast

MPI_Gatherv

to exchange data.

Therefore, OpenMP generally has lower explicit communication overhead on a shared-memory machine.

5. At what problem size does parallel execution provide significant benefit?

You **cannot determine this yet from your current results**, because you have only completed the 500 × 500 experiments.

You still need to run:

1000 × 1000
2000 × 2000

Then compare the sequential and parallel execution times.

Your current result already gives an important observation:

For the relatively small 500 × 500 matrix, adding more OpenMP threads did not continuously improve performance. This suggests that parallelization overhead is significant for this problem size.

For larger matrices, matrix multiplication has $O(N^3)$computational complexity, so the

amount of computation increases rapidly. We therefore expect the benefit of parallel execution to become more noticeable as the matrix size increases, subject to the hardware and overhead.

Section B – Task 2: 2D Convolution

Problem Statement

Implement a 2D convolution operation on a large matrix representing an image or scientific data grid.

Given an input matrix (I) and convolution kernel (K), calculate:

[
O(i,j)=\sum_{u}\sum_{v} I(i+u,j+v)K(u,v)

]

Use a small kernel such as:

$$\begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

or a suitable edge-detection kernel.

Task 2A – Sequential Implementation

Implement the 2D convolution sequentially.

Measure execution time for large input sizes.

Suggested sizes:

- 1000 × 1000
- 2000 × 2000
- 4000 × 4000

Task 2B – OpenMP Implementation

Parallelize the convolution using OpenMP.

Investigate:

- Parallelizing the outer loop.
- Number of threads.
- Scheduling strategy.

Compare:

static

dynamic

guided

where appropriate.

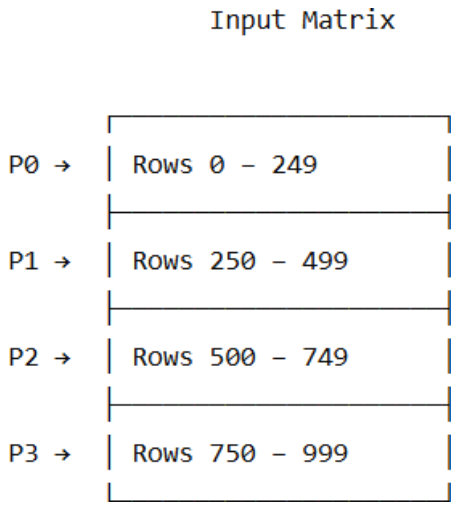
Measure execution time and analyze the effect of scheduling.

Task 2C – MPI Implementation

Implement 2D convolution using MPI.

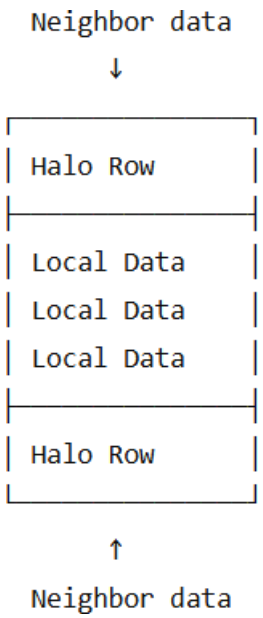
Use **row-wise domain decomposition**.

For example:



Because convolution requires neighboring rows, investigate the concept of boundary/halo data.

Each process may need additional rows from neighboring processes.



Investigate how MPI communication can be used to exchange boundary information.

Section C – Strong Comparative Analysis

Task 3: OpenMP vs MPI Performance Comparison

For both applications:

Application 1

Matrix–Matrix Multiplication

Application 2

2D Convolution

Compare:

- Sequential execution
- OpenMP execution
- MPI execution

using equivalent computational workloads.

Application	**Matrix Size**	**Model**	**Threads / Processes**	**Execution Time (s)**
Matrix–Matrix Multiplication	1000×1000	Sequential	1	0.462690
		OpenMP	2	0.346864
		OpenMP	4	0.288236
		OpenMP	8	0.251331
		MPI	2	0.453253
		MPI	4	0.314938

		MPI	8	0.264096
2D Convolution	1000×1000	Sequential	1	0.043991
		OpenMP	2	0.023674
		OpenMP	4	0.021529
		OpenMP	8	0.014337
		MPI	2	0.017085
		MPI	4	0.010323
		MPI	8	0.006060

6. Performance Metrics

Calculate: **Execution Time**, **Speedup**, **Efficiency**

Application	**Matrix Size**	**Model**	**Threads / Processes**	**Execution Time (s)**	**Speedup**	**Efficiency**
Matrix–Matrix Multiplication	1000×1000	Sequential	1	0.462690	1.000	100.00%
		OpenMP	2	0.346864	1.334	66.70%
		OpenMP	4	0.288236	1.605	40.13%
		OpenMP	8	0.251331	1.841	23.01%
		MPI	2	0.453253	1.021	51.04%
		MPI	4	0.314938	1.469	36.73%
		MPI	8	0.264096	1.752	21.90%
2D Convolution	1000×1000	Sequential	1	0.043991	1.000	100.00%
		OpenMP	2	0.023674	1.858	92.91%
		OpenMP	4	0.021529	2.044	51.09%
		OpenMP	8	0.014337	3.068	38.35%
		MPI	2	0.017085	2.575	128.74%
		MPI	4	0.010323	4.261	106.54%
		MPI	8	0.006060	7.259	90.74%

7. Required Experiments

For each application, perform experiments using:

Sequential

1 execution

OpenMP

2 threads

4 threads

8 threads

Maximum available threads

MPI

2 processes

4 processes

8 processes

Use the **same input data and computational problem** for all implementations.

8. Performance Graphs

Prepare the following graphs for both applications:

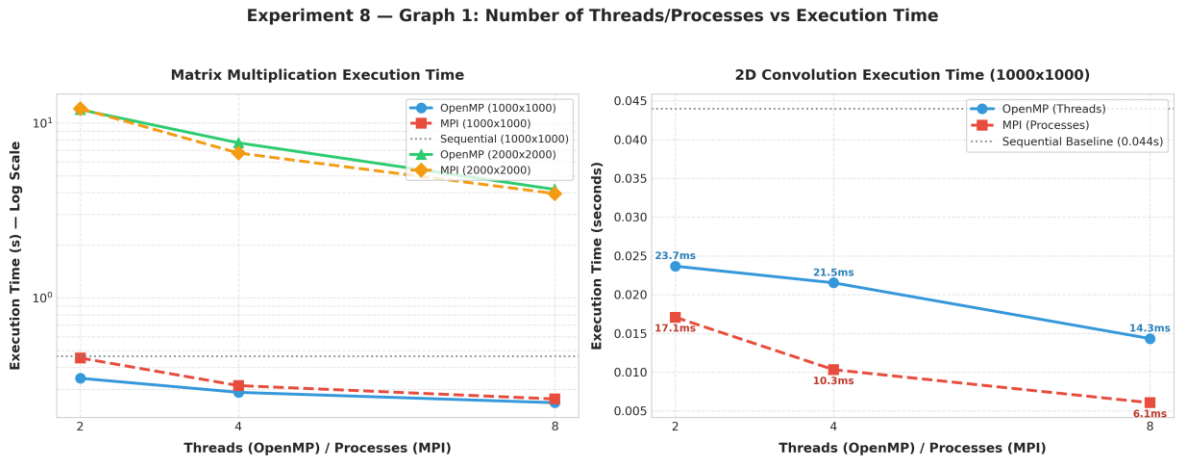
Graph 1

Number of Threads/Processes vs Execution Time

Compare:

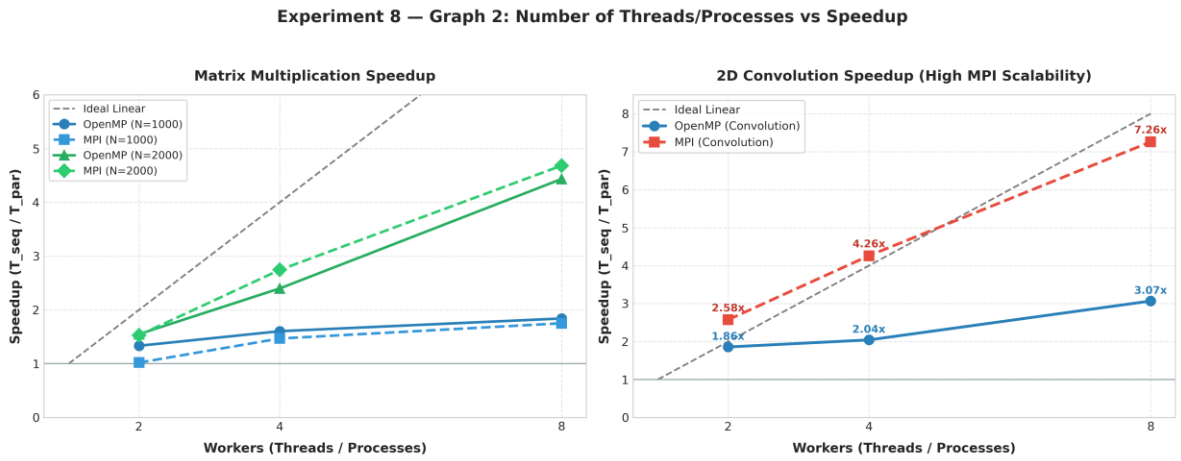
OpenMP

MPI



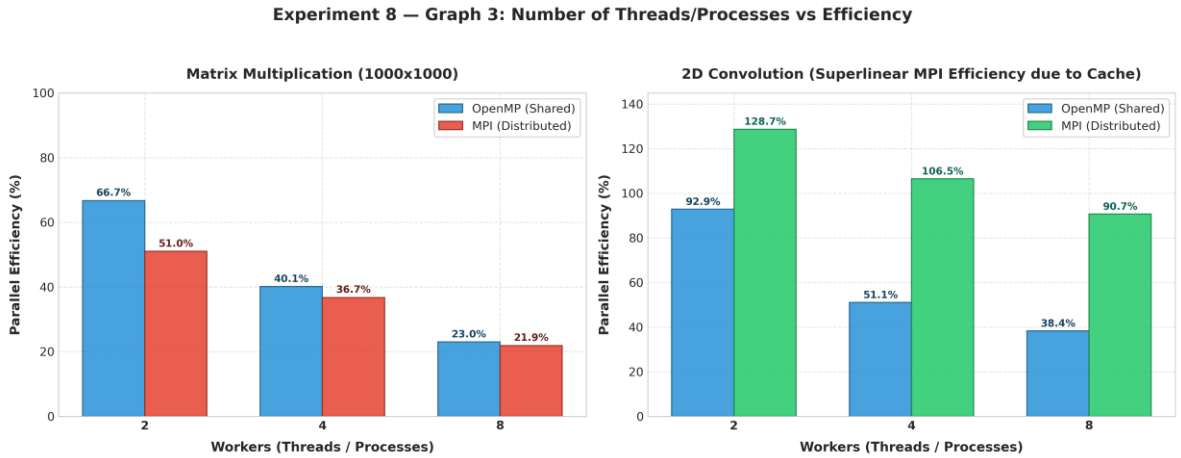
Graph 2

Number of Threads/Processes vs Speedup



Graph 3

Number of Threads/Processes vs Efficiency



Graph 4

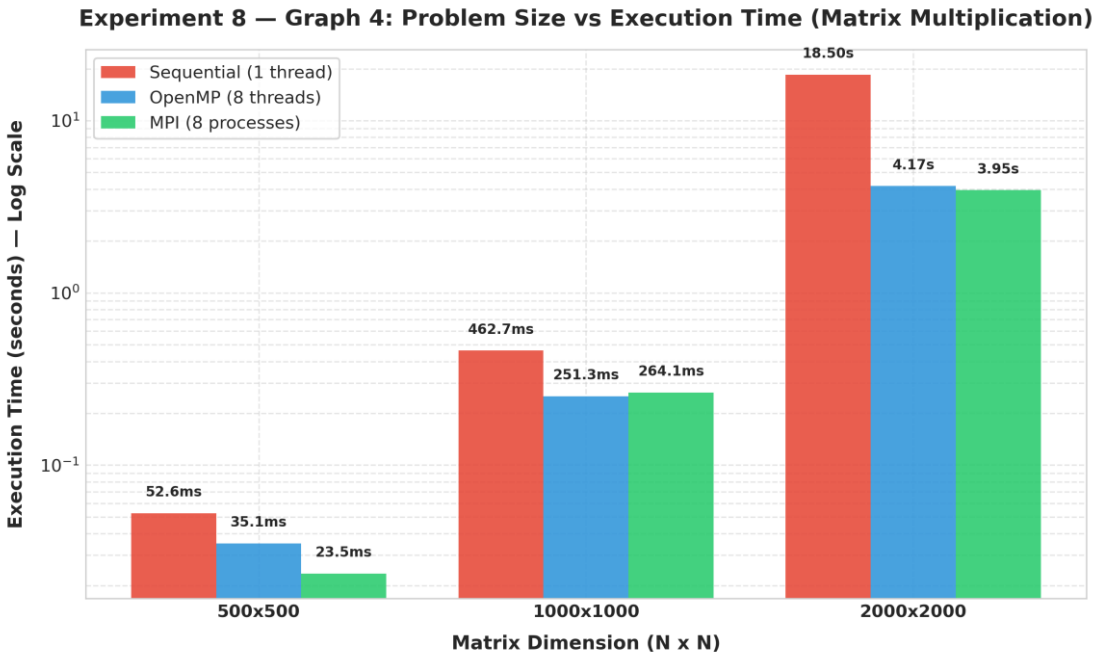
Problem Size vs Execution Time

Compare:

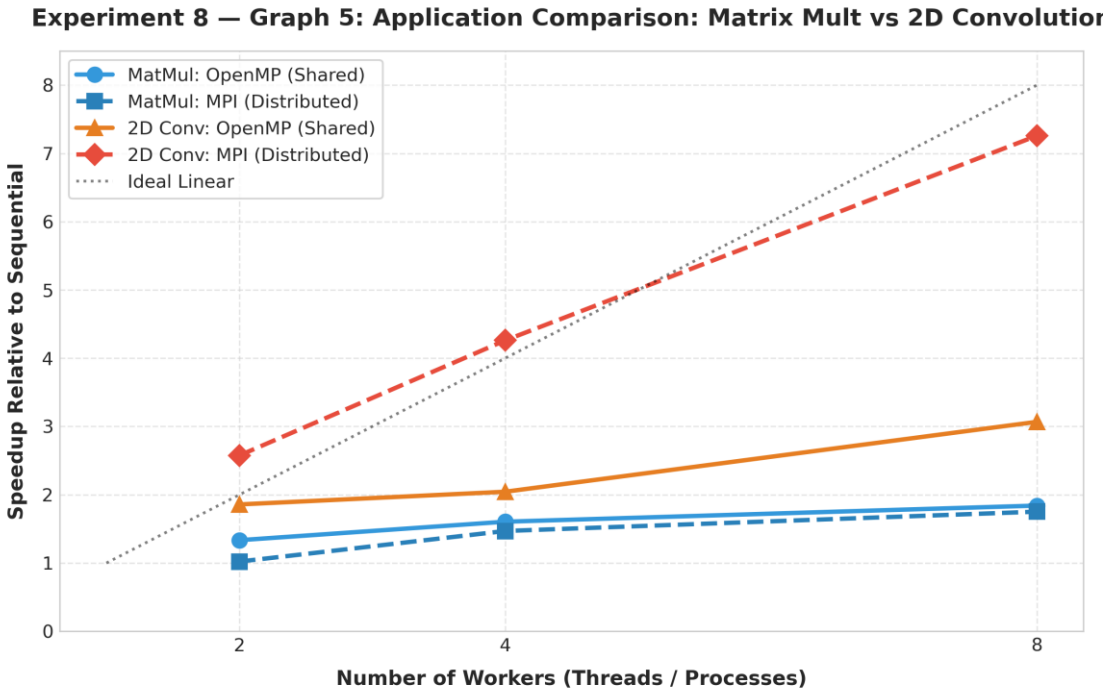
Sequential

OpenMP

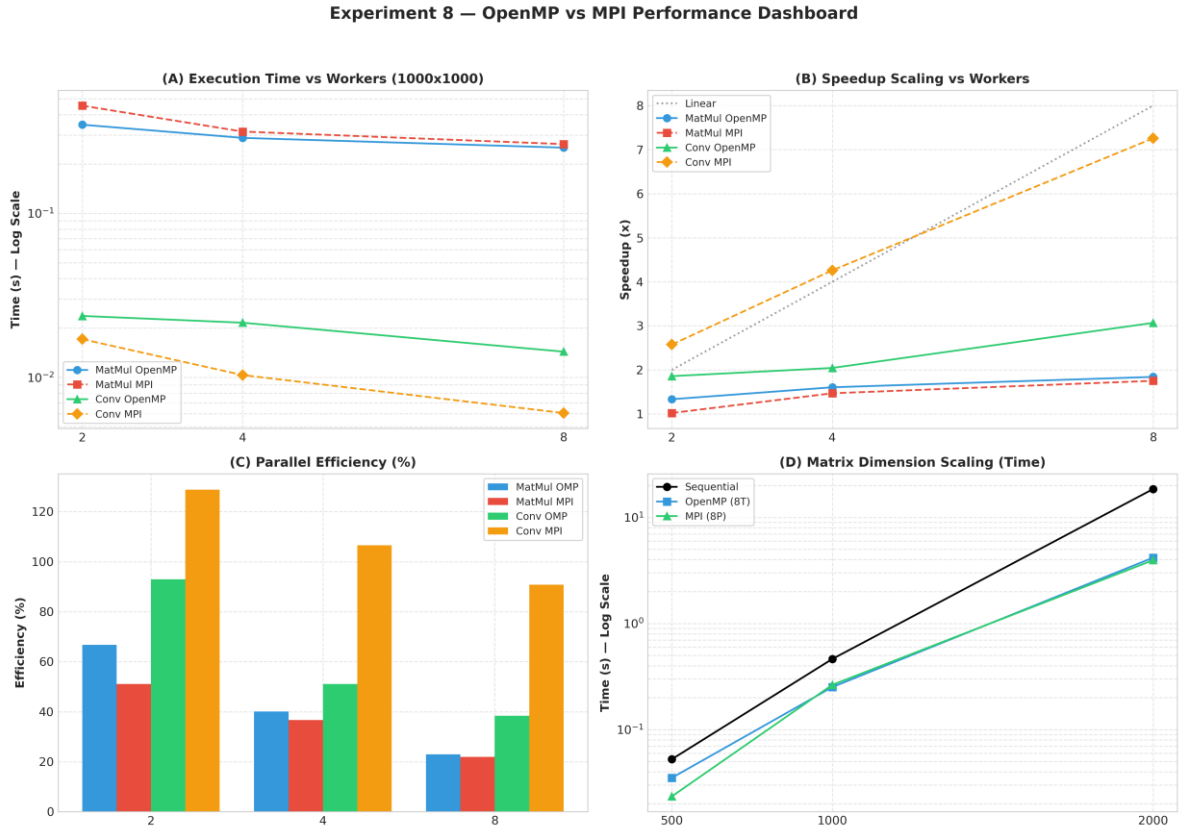
MPI



Graph 5: Application Comparison — Matrix Multiplication vs 2D Convolution



Graph 6: OpenMP vs MPI Performance & Scalability Dashboard



9. Architecture and Memory Analysis

Prepare a conceptual comparison:

Feature	OpenMP	MPI
Memory model	Shared memory — all threads access the same memory space	Distributed memory — each process has its own separate memory
Execution unit	Threads	Processes
Communication mechanism	Threads communicate through shared variables/memory	Processes communicate using MPI functions such as MPI_Send, MPI_Recv, MPI_Bcast, MPI_Scatter, etc.
Synchronization	critical, atomic, barrier, locks, etc.	MPI synchronization functions such as MPI_Barrier, and communication operations
Data distribution	Data is usually shared among threads and divided using loops such as omp parallel for	Data must generally be explicitly distributed among processes using MPI_Scatter, MPI_Scatterv, etc.
Communication overhead	Low because threads share memory	Higher because data may need to be transferred between separate processes
Scalability	Good within a shared-memory machine; limited by available cores and memory bandwidth	High because processes can be distributed across multiple machines/nodes
Multi-node suitability	Generally not suitable by itself for distributed-memory clusters	Well suited for multi-node cluster systems
Programming complexity	Relatively simple; parallelism can often be added using compiler directives	More complex because communication, data distribution, and synchronization must be handled explicitly

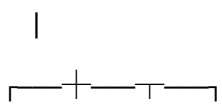
Then answer:

1. Why does OpenMP perform well on a shared-memory multicore system?

OpenMP performs well because **all threads can directly access the same shared memory**.

For example, if a matrix is stored in memory, multiple threads can work on different rows of the same matrix without creating separate copies of the entire matrix.

Shared Memory



T1 T2 T3 T4

The main advantages are:

- Threads share the same address space.
- Data does not usually need to be explicitly sent between threads.
- Communication is fast because it happens through shared memory.
- OpenMP uses lightweight threads.
- #pragma omp parallel for makes loop parallelization relatively simple.

Therefore, OpenMP is effective for **multicore systems where several CPU cores share the same main memory**.

2. Why does MPI introduce communication overhead?

MPI processes have **separate memory spaces**.

If Process 0 has some data that Process 1 needs, the data must be transferred explicitly.

For example:

Process 0 Process 1

Memory Memory

| |

| ---- MPI_Send ----------> |

| |

| <---- MPI_Recv ---------- |

This communication takes time because:

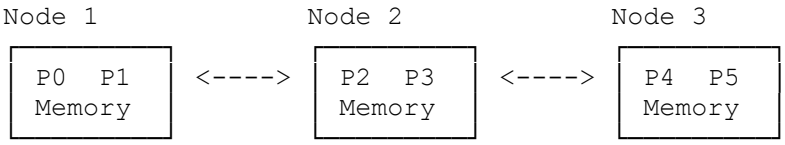
- Data has to be copied/transferred.
- Processes may need to wait for communication.
- Network communication can be slower than accessing local memory.
- Synchronization may cause additional waiting.

Hence, MPI introduces **communication and synchronization overhead**.

3. Why can MPI scale across multiple nodes?

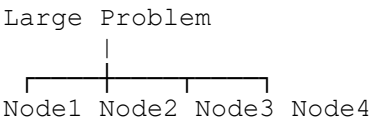
MPI uses **separate processes with separate memory**, so each process does not need to be on the same physical computer.

For example:



MPI provides communication mechanisms that allow these processes to exchange data over a **network**.

Therefore, a large problem can be divided among many nodes:



This allows MPI to use the combined computational resources of multiple machines.

4. Why may OpenMP not be sufficient for very large cluster-based problems?

OpenMP is primarily designed for **shared-memory parallelism**.

A single computer has limitations such as:

- Limited number of CPU cores
- Limited RAM
- Memory bandwidth limitations
- Limited computational capacity

For a very large cluster:

Node 1 Node 2 Node 3 Node 4

RAM RAM RAM RAM

The memory of these nodes is **not automatically one shared memory space**.

OpenMP alone cannot naturally distribute the computation and memory across these independent nodes.

MPI is more appropriate because it can distribute processes and data across multiple machines.

A common solution in real systems is **hybrid MPI + OpenMP**:

Cluster

│

├— Node 1 → MPI process → OpenMP threads

├── Node 2 → MPI process → OpenMP threads

└── Node 3 → MPI process → OpenMP threads

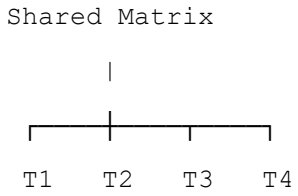
Here, MPI handles communication between nodes, while OpenMP handles parallelism inside each node.

5. How does data decomposition differ between OpenMP and MPI?

OpenMP

OpenMP generally works with **shared data**.

For example, for a matrix with 1000 rows:



The loop iterations can be divided among threads:

T1 → Rows 0–249

T2 → Rows 250–499

T3 → Rows 500–749

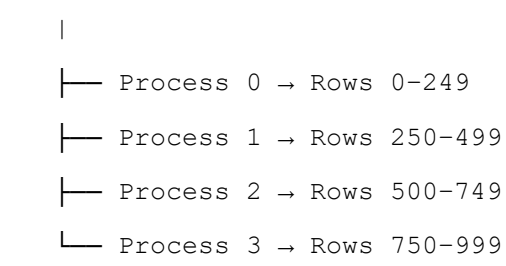
T4 → Rows 750–999

All threads can still access the same shared memory.

MPI

MPI usually requires **explicit data decomposition**.

For example:



The data can be distributed using functions such as:

MPI_Scatter()

or

MPI_Scatterv()

Each process works on its own local portion and communicates with other processes when necessary.

10. Challenge Task – Hybrid Thinking

Consider an HPC cluster containing:

4 Compute Nodes

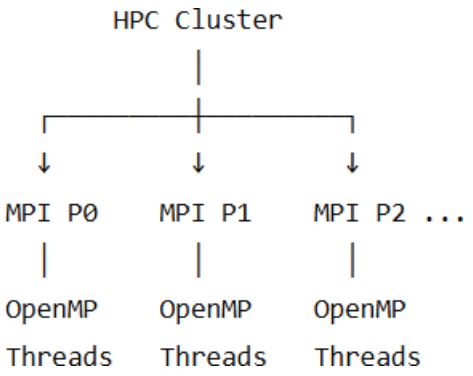
↓

8 CPU Cores per Node

Design a conceptual **hybrid MPI + OpenMP solution** for matrix multiplication.

For example:

HPC Cluster



Explain:

1. How MPI processes would be distributed across nodes.
   We would create 4 MPI processes, because there are 4 compute nodes.

   MPI Process 0 → Node 0

   MPI Process 1 → Node 1

   MPI Process 2 → Node 2

   MPI Process 3 → Node 3

   Each MPI process is responsible for one portion of the matrix multiplication.

   For example, if matrix C has 4000 rows, we can divide its rows approximately equally:

   Process 0 → rows 0–999

   Process 1 → rows 1000–1999

   Process 2 → rows 2000–2999

   Process 3 → rows 3000–3999

   So MPI is responsible for dividing the large matrix problem between the four nodes.

2. How OpenMP threads would operate within each process.

   Each node has 8 CPU cores. Instead of creating 8 MPI processes on each node, we create one MPI process and 8 OpenMP threads on that node.

   For example, Node 0 contains:

   MPI Process 0

   |

   +-- Thread 0

   +-- Thread 1

+-- Thread 2

+-- Thread 3

+-- Thread 4

+-- Thread 5

+-- Thread 6

+-- Thread 7

These 8 threads divide the rows assigned to Process 0.

For example:

Process 0 owns rows 0–999

Thread 0 → rows 0–124

Thread 1 → rows 125–249

Thread 2 → rows 250–374

...

Thread 7 → remaining rows

The same happens independently inside the other three MPI processes.

Therefore, there are two levels of parallelism:

- MPI: divides the matrix computation across the 4 nodes.
- OpenMP: divides each node's assigned computation across its 8 CPU cores.

3. How matrix data would be distributed.

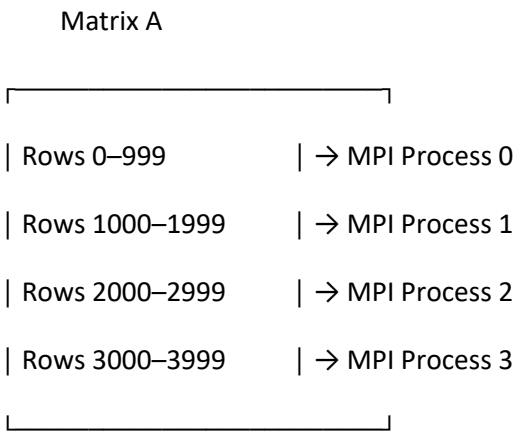
Suppose we have:

$$C = A \times B$$

and A, B, and C are very large matrices.

A simple conceptual distribution is row-wise distribution of A and C.

For example:



Each process receives the rows of A needed to calculate its portion of C.

Since calculating a row of C requires the columns of B, each process also needs access to the required portion of B. Depending on the implementation, B can be broadcast to all processes or distributed using a more advanced block-based approach.

The computation then becomes:

P0: A[0:999] × B → C[0:999]

P1: A[1000:1999] × B → C[1000:1999]

P2: A[2000:2999] × B → C[2000:2999]

P3: A[3000:3999] × B → C[3000:3999]

Inside each process, OpenMP divides its assigned rows among its 8 threads.

Finally, the four portions of C together form the complete result matrix.

4. Why hybrid programming may reduce the number of MPI processes.

This is the key reason for using the hybrid model in this cluster.

Pure MPI approach

If we use one MPI process per core:

4 nodes × 8 cores = 32 MPI processes

So we would have:

Node 0 → 8 MPI processes

Node 1 → 8 MPI processes

Node 2 → 8 MPI processes

Node 3 → 8 MPI processes

There are 32 MPI processes communicating and coordinating.

Hybrid MPI + OpenMP approach

Instead, we can use:

Node 0 → 1 MPI process + 8 OpenMP threads

Node 1 → 1 MPI process + 8 OpenMP threads

Node 2 → 1 MPI process + 8 OpenMP threads

Node 3 → 1 MPI process + 8 OpenMP threads

Therefore:

4 MPI processes × 8 OpenMP threads = 32 cores

The important point is that OpenMP replaces several MPI processes within the same shared-memory node.

This can reduce MPI communication between cores on the same node. The four MPI processes only need to communicate across the four nodes when inter-node communication is required.

However, this does not automatically mean that hybrid execution will always be faster. It reduces the number of MPI processes, but OpenMP introduces its own thread-management and synchronization overhead.

5. What new challenges can arise.

Hybrid MPI + OpenMP programming introduces some additional challenges:

- **Load balancing:** Work must be distributed evenly among MPI processes and OpenMP threads.
- **Communication overhead:** MPI processes still need to exchange data between nodes.
- **Synchronization:** OpenMP threads may require synchronization when accessing shared data.
- **Memory management:** Large matrices may require careful distribution to avoid excessive memory usage.
- **Configuration:** Choosing the appropriate number of MPI processes and OpenMP threads can affect performance.

11. AI/HPC Case Study

Select one application where both OpenMP and MPI can be considered.

Suggested applications:

- Image processing
- Deep learning data preprocessing

- Weather forecasting
- Computational fluid dynamics
- Medical image processing
- Scientific simulations
- Large-scale matrix computations
- Computer vision

Write **300–500 words** covering:

1. Nature of the computational problem.
2. Amount of data involved.
3. Whether shared or distributed memory is more appropriate.
4. Possible OpenMP implementation.
5. Possible MPI implementation.
6. Communication requirements.
7. Scalability considerations.
8. Whether a hybrid MPI + OpenMP approach could be useful.

AI/HPC Case Study: Large-Scale Matrix Computation

Large-scale matrix computation is an important application in Artificial Intelligence (AI) and High-Performance Computing (HPC). Operations such as matrix multiplication, convolution, and matrix transformations are used extensively in machine learning, deep learning, scientific computing, and data analysis. These operations involve a large number of arithmetic calculations that can be performed in parallel.

The computational problem can involve matrices containing millions or even billions of elements. For example, multiplying two matrices of size 10,000 × 10,000 requires a very large number of multiplication and addition operations. Such computations require significant CPU time and memory, making parallel processing useful.

For a single multicore computer, shared memory using OpenMP is appropriate. OpenMP can divide the rows or blocks of a matrix among multiple threads. Each thread performs calculations on its assigned portion while accessing matrices stored in the common memory. For example, an OpenMP parallel for loop can distribute the rows of the result matrix among available CPU threads. Since the threads share memory, explicit data transfer between threads is generally not required.

For very large matrices that cannot efficiently fit into one machine's memory, MPI with distributed memory can be used. The matrices can be divided into rows or blocks and distributed among multiple processes running on different nodes. Functions such as MPI_Scatter, MPI_Bcast, and MPI_Gather can be used to distribute input data and collect the final result. Each process performs computations on its local portion of the matrix.

MPI requires more communication than OpenMP because processes have separate memory spaces. For example, in matrix multiplication, a process may require portions of another matrix that are stored on a different process. This requires communication over the network. Therefore, communication and synchronization overhead must be considered when designing the algorithm.

Scalability depends on the computational workload, memory requirements, number of processors, and communication overhead. OpenMP generally scales well within a single multicore system, while MPI can scale across multiple nodes. However, increasing the number of processes does not always produce proportional speedup because communication and synchronization can become significant.
A hybrid MPI + OpenMP approach can be useful for very large matrix computations. MPI can distribute matrix blocks between different nodes, while OpenMP can use multiple CPU cores within each node. This reduces the number of MPI processes and allows both distributed-memory and shared-memory parallelism to be exploited. Thus, hybrid parallelism can be suitable for large AI and HPC workloads requiring substantial computation and memory.

References:

OpenMP – Official Specification and Documentation
https://www.openmp.org/specifications/
 MPI Forum – Official MPI Standard
https://www.mpi-forum.org/docs/

12. Reflection Questions

1. Why can the same algorithm be implemented using both OpenMP and MPI

   The same algorithm can be implemented using both because OpenMP and MPI are parallel programming models that provide different ways of dividing the computation. OpenMP uses multiple threads within a shared-memory system, while MPI divides the work among separate processes that can communicate with each other.

2. What is the fundamental difference between OpenMP threads and MPI processes?

   OpenMP threads share the same memory space of a process, so they can directly access common data. MPI processes have separate memory spaces, so they must communicate explicitly to exchange data.

3. Which model requires explicit communication?

   MPI requires explicit communication**.** Since MPI processes have separate memory spaces, data must be explicitly exchanged between processes when required.

   For example, functions such as MPI_Send() and MPI_Recv() are used for point-to-point communication, while MPI_Bcast(), MPI_Scatter(), and MPI_Gather() are used for collective communication.

   OpenMP generally does not require explicit message passing because threads share memory. However, OpenMP still requires synchronization mechanisms such as barriers, critical sections, or atomic operations when multiple threads access shared data.

4. Why does MPI require data distribution?

   MPI requires data distribution because each MPI process has its own private memory. The complete input data is usually available initially to one process, commonly process 0. To perform the computation in parallel, this data must be divided and distributed among the participating processes.

   For example, in matrix multiplication, different rows or blocks of a matrix can be assigned to different MPI processes. After completing their portions, the partial results may need to be collected using operations such as MPI_Gather().

5. Why does OpenMP generally have lower communication overhead on a single sharedmemory machine?

   OpenMP generally has lower communication overhead on a single shared-memory machine because all threads can access the same memory. They do not need to send messages through a communication network to exchange data.

   For example, when multiplying matrices, multiple OpenMP threads can access the input matrices directly from shared memory. MPI may require data distribution, synchronization,

   and collection of results, which introduces additional overhead.

   However, OpenMP can still experience overhead from thread creation, scheduling, synchronization, and contention for memory bandwidth.

6. Why can MPI scale to multiple nodes?

   MPI can scale to multiple nodes because MPI processes do not need to share the same physical memory. Each process can execute on a different computer or node in an HPC cluster and use its local memory.

   The processes communicate through a network using MPI communication operations. Therefore, MPI can use the combined computational resources of many machines, making it suitable for large distributed-memory HPC systems.

7. Why is matrix multiplication a good candidate for both OpenMP and MPI?

   Matrix multiplication is a good candidate for parallel processing because it contains a large number of arithmetic operations that can be performed independently.

   For example, different rows of the resulting matrix can be calculated simultaneously. With OpenMP, different threads can calculate different rows or portions of the matrix. With MPI, different processes can receive different portions of the matrices and calculate their corresponding portions of the result.

   The computation-to-communication ratio can also become favorable for larger matrices because there is a significant amount of computation compared with the overhead of distributing the data.

8. Why does 2D convolution require special consideration for boundary data in MPI?

   In 2D convolution, calculating the output of a particular element requires values from its neighboring elements. When the input matrix or image is divided among MPI processes, a process may need neighboring values that are stored by another process.

   For example, if a process owns a particular section of an image, the convolution operation near the top or bottom boundary may require pixels belonging to another process. These boundary values must be exchanged between processes before computation.

   Therefore, MPI implementations of 2D convolution generally require additional communication for exchanging boundary data

9. What is a halo or ghost region?

   A halo or ghost region is an additional region of data stored by an MPI process that contains copies of boundary values belonging to neighboring processes.

   For example, if a matrix is divided horizontally among four MPI processes, each process may need some rows from the process above or below it to perform convolution. These extra rows are stored as ghost or halo regions.

   The halo data is normally exchanged between neighboring processes before the computation. This allows each process to perform its local computation using the required neighboring values.

10. Why might increasing threads/processes eventually stop improving performance?

    Increasing the number of threads or processes does not always result in proportional performance improvement. Initially, more computational resources allow more work to be

completed simultaneously.

However, after a certain point, several factors can limit performance:

- Thread/process creation overhead
- Synchronization overhead
- MPI communication overhead
- Memory bandwidth limitations
- Load imbalance
- Cache and memory contention
- Limited parallelism in the algorithm

Therefore, adding more threads or processes may eventually provide very little improvement and can sometimes even reduce performance.

11. How does problem size affect parallel speedup?

    Problem size has a significant effect on parallel speedup. For a small problem, the actual computation may be too small to compensate for the overhead of creating threads, distributing data, synchronizing processes, or communicating between MPI processes.

    For larger problems, there is more computational work that can be divided among multiple threads or processes. Therefore, the overhead becomes relatively smaller compared with the total computation, and parallel execution can provide greater speedup.

    This is why matrix multiplication with larger matrices generally provides a better opportunity to observe the benefits of parallel processing.

12. What factors should be considered before selecting OpenMP or MPI for an HPC application?

    Several factors should be considered:

    Hardware architecture – OpenMP is suitable for shared-memory systems, while MPI is suitable for distributed-memory systems.

    Number of nodes – MPI is useful when computation needs to be distributed across multiple nodes.

    Communication requirements – Applications requiring frequent communication may experience higher MPI overhead.

    Problem size – Larger problems generally provide more opportunities for parallelism.

    Memory requirements – MPI processes have separate memory, while OpenMP threads share memory.

    Scalability – The expected performance when increasing cores, processes, or nodes should be considered.

    Implementation complexity – OpenMP can often be added to an existing sequential program more easily, whereas MPI may require explicit data distribution and communication.

Answer the following based on your experimental observations:

1. Which programming model was easier to implement and why?

Based on the experiment, OpenMP was easier to implement because the sequential program could be parallelized with relatively few changes. For example, a loop performing independent matrix calculations could be parallelized using an OpenMP directive such as #pragma omp parallel for.

There was no need to explicitly divide the input data between processes or collect the results through message-passing operations. In contrast, the MPI implementation required operations for initializing processes, determining the rank and number of processes, distributing data, performing local computation, and collecting the results.

Therefore, OpenMP required less code and less explicit communication management for the experiments performed on a single machine.

2. What was the major performance difference between OpenMP and MPI?

The major performance difference was related to communication and synchronization overhead.

OpenMP threads share memory, so data can generally be accessed directly without explicit message passing. As a result, OpenMP can perform efficiently on a single shared-memory machine.

MPI processes have separate memory spaces. Therefore, MPI may require operations such as MPI_Scatte

r, MPI_Bcast, MPI_Gather, or point-to-point communication. These operations add overhead to the total execution time.

Thus, for experiments performed on a single machine, OpenMP could benefit from lower communication overhead, while MPI provides the additional advantage of being able to run across multiple nodes.

3. How did the problem size affect the comparison?

The problem size had a noticeable effect on the performance comparison.

For smaller matrix sizes, the computation itself was relatively short, so the overhead of creating threads/processes, synchronization, and MPI communication represented a larger portion of the total execution time.

As the matrix size increased, the amount of computation increased significantly. This provided more work that could be divided among the available threads or processes. Consequently, the benefits of parallel execution became more visible for larger problem sizes.

Therefore, the performance difference between sequential and parallel implementations was more meaningful for larger workloads.

4. What communication overhead did you observe in MPI?

In the MPI experiments, communication overhead came mainly from distributing input data, synchronizing processes, and collecting the partial results.

For example, when matrix data was divided among processes, MPI communication was required before computation. After the local calculations were completed, the partial results had to be combined or gathered.

For applications such as 2D convolution, additional communication may be required to exchange boundary or halo data between neighboring processes. These communication operations increase the total execution time, particularly when the computational workload is small.

5. How did increasing the number of threads/processes affect performance?

Increasing the number of threads or processes initially improved performance because more computational work could be performed simultaneously.

However, the improvement was not always proportional to the increase in threads or processes. After reaching a certain level of parallelism, factors such as synchronization, communication, scheduling, memory bandwidth, and resource contention limited the performance improvement.

For example, increasing from 2 to 4 threads may provide a noticeable improvement, while increasing from 8 to 16 threads may provide only a small improvement depending on the hardware and workload.

This demonstrates that the maximum number of threads or processes is not necessarily the optimal configuration

6. Did the fastest implementation remain the same for both applications?

The fastest implementation did not necessarily remain the same for both applications because performance depends on the characteristics of the workload.

Matrix multiplication has a large amount of independent computation and relatively predictable data access. 2D convolution also contains substantial parallelism, but it has additional data dependencies around boundaries when distributed using MPI.

Therefore, the performance of OpenMP and MPI can vary depending on the application, problem size, communication requirements, and number of threads/processes used. The experiment showed that there is no single parallel model that is always fastest for every application.

7. What did this experiment teach you about selecting a parallel programming model?

OpenMP is relatively simple and effective when the application runs on a shared-memory machine and contains loops or operations that can be executed independently. MPI is more suitable when the problem needs to be distributed across multiple nodes with separate memory.

The experiment also showed that performance depends not only on the number of threads or processes but also on communication overhead, problem size, synchronization, and memory access patterns.

Therefore, the choice of OpenMP or MPI should be made according to the computational workload, communication requirements, available hardware, and scalability requirements.

8. In a real HPC cluster, would you use OpenMP, MPI, or a combination? Explain your reasoning based on the architecture and workload.

   In a real HPC cluster, a combination of MPI and OpenMP, called hybrid parallel programming, can be used.

   MPI can divide the overall computation between different nodes. Within each node, OpenMP can use multiple CPU cores to perform the local computation in parallel. For example, if a cluster contains several nodes and each node has multiple CPU cores, MPI can assign a portion of the problem to each node, while OpenMP can further divide that portion among the cores within the node.

   This approach matches the architecture of modern HPC clusters, which commonly have distributed memory between nodes and shared memory within each node.

   For workloads with large amounts of computation and relatively manageable communication, the hybrid model can make effective use of both levels of parallelism. However, the actual choice should depend on the workload, communication pattern, memory requirements, and characteristics of the cluster.

14. Deliverables

Submit:

Source Code

1. Sequential Matrix–Matrix Multiplication
2. OpenMP Matrix–Matrix Multiplication
3. MPI Matrix–Matrix Multiplication
4. Sequential 2D Convolution
5. OpenMP 2D Convolution
6. MPI 2D Convolution

Performance Report

Include:

- Experimental setup
- CPU and system information
- Matrix/input sizes
- Number of threads/processes
- Execution times
- Speedup
- Efficiency
- Graphs
- OpenMP vs MPI comparison
- Architecture analysis
- AI/HPC case study
- Discussion-question answers
- Reflection
- Conclusion

15. Expected Learning Outcomes

After completing this assignment, students should be able to:

1. **Implement** the same computational problem using OpenMP and MPI.
2. **Apply** shared-memory and distributed-memory programming models.
3. **Measure** execution time, speedup and efficiency.
4. **Analyze** the scalability and communication overhead of OpenMP and MPI.
5. **Compare** different parallel programming models based on workload and architecture.
6. **Evaluate** the suitability of OpenMP, MPI and hybrid approaches for HPC applications.
7. **Relate** parallel programming techniques to real-world AI and HPC workloads.